# Day 3 학습 정리 — 2026-06-28 시작 / 2026-06-29 완료 ✅

## 한 것

- `Child` 모델에 `lesson_records` relationship 추가
- `lesson_records_router.py` 생성 및 `POST /lesson-records` 구현
- `GET /lesson/{child_id}/lesson-records` 구현 — 특정 아이의 최근 수업기록 5개 조회 (filter + order_by + limit)
- `main.py`에 lesson_records 라우터 등록
- 🐞 **버그 3개 잡고** 테스트 데이터로 동작 확인 (아래 디버깅 섹션 참고)

---

## 핵심 개념

### relationship 양방향 연결

두 테이블이 서로를 참조할 때 양쪽 모두에 `relationship`을 써야 해.

```
# models.py
class Child(Base):
    ...
    lesson_records = relationship("LessonRecord", back_populates="child")
    #  ↑ "이 아이의 수업 기록 목록"

class LessonRecord(Base):
    ...
    child = relationship("Child", back_populates="lesson_records")
    #  ↑ "이 기록의 아이 정보"
```

한쪽만 있으면 서버 켤 때 에러 남.

---

### GET / vs GET /{id} 차이

| | `GET /` | `GET /{id}` |
|---|---|---|
| 조건 | 없음 | id 일치하는 것 |
| 결과 | 전체 리스트 | 하나 |
| 끝에 붙는 것 | `.all()` | `.first()` |

```
# 전체 조회
db.query(Child).all()

# 특정 하나 조회
```

```
db.query(Child).filter(Child.id == child_id).first()

# 특정 아이의 수업 기록 전체 조회 (리스트)
db.query(LessonRecord).filter(LessonRecord.child_id == child_id).all()
```

---

## 파라미터(parameter)란?

함수가 받는 입력값.

```
def create_LessonRecord(record: LessonRecordCreate, db: Session):
#                       ↑ 파라미터
```

**이름 충돌 주의:**

```
from models import LessonRecord  # 클래스 이름

# 잘못된 예 — 파라미터 이름이 클래스 이름과 같음
def create(LessonRecord: LessonRecordCreate):
    new = LessonRecord(...)  # 클래스인지 파라미터인지 Python이 헷갈림

# 올바른 예
def create(record: LessonRecordCreate):
    new = LessonRecord(...)  # 클래스
    new.child_id = record.child_id  # 파라미터에서 꺼냄
```

---

## POST /lesson-records 패턴

```
@router.post("/", response_model=LessonRecordResponse)
def create_LessonRecord(record: LessonRecordCreate, db: Session =
Depends(get_db)):
    new_LessonRecord = LessonRecord(   # 모델 클래스로 객체 생성
        child_id=record.child_id,
        lesson_date=record.lesson_date,
        activity=record.activity,
        performance=record.performance,
        attitude=record.attitude,
        difficulty=record.difficulty,
        teacher_note=record.teacher_note
        # created_at은 DB가 자동으로 넣어줌 — 여기 쓰지 않음
    )
    db.add(new_LessonRecord)
    db.commit()
    db.refresh(new_LessonRecord)
    return new_LessonRecord
```

**`created_at`을 파라미터로 받지 않는 이유:** `models.py`에서 `server_default=func.now()`로 DB가 자동 입력하게 설정되어 있어서, 사용자가 보내는 게 아님.

---

## 파일을 나누는 이유 (main.py vs router 파일)

```
main.py
  ├── children_router.py        (아이 관련 API)
  ├── lesson_records_router.py  (수업 기록 관련 API)
  └── feedbacks_router.py       (피드백 관련 API)  ← Day 5
```

전부 main.py에 넣으면 수백 줄이 돼서 관리 불가. main.py는 앱을 켜고 라우터를 연결만 함.

---

## 조건 조회: filter + order_by + limit 체이닝 (2026-06-29)

"특정 아이의 최근 수업기록 5개"처럼 **일부만** 조회할 땐 `db.query(...)` 뒤에 메서드를 점(`.`)으로 이어붙인다(체이닝).

```python
from sqlalchemy import desc   # desc(내림차순)는 import 필요!

@router.get("/{child_id}/lesson-records")
def get_lesson_records(child_id: int, db: Session = Depends(get_db)):
    records = (
        db.query(LessonRecord)
          .filter(LessonRecord.child_id == child_id)   # 이 아이 것만
          .order_by(desc(LessonRecord.lesson_date))    # 최신순
          .limit(5)                                    # 5개까지
          .all()                                       # 리스트로 꺼내기
    )
    return records
```

| 메서드 | 역할 |
|---|---|
| `.filter()` | 조건으로 거르기 |
| `.order_by(desc(...))` | 정렬 (desc=내림차순=최신 먼저) |
| `.limit(n)` | 개수 제한 |
| `.all()` | 결과를 리스트로 꺼냄 (맨 끝) |

⭐ **제일 많이 헷갈린 부분 — `filter(왼쪽 == 오른쪽)`:**

```python
.filter(LessonRecord.child_id == child_id)
#       └ 왼쪽: 테이블 컬럼 ┘     └ 오른쪽: 들어온 값 ┘
```

- **왼쪽** `LessonRecord.child_id` = DB 테이블에 **저장된 컬럼**
- **오른쪽** `child_id` = URL로 **들어온 값**(함수 파라미터). 컬럼이 아니라 그냥 숫자(예: 3)!
- 말로 풀면: "테이블의 child_id **컬럼**이, 들어온 **값** 3과 같은 행을 찾아라"
- 내가 했던 실수: `Child.id`(다른 테이블) → `child_id == child_id`(값끼리 비교, 항상 참) → 정답 `LessonRecord.child_id == child_id`

**참고:** 이 라우터는 `prefix="/lesson"`이라 실제 주소는 `/lesson/{child_id}/lesson-records`가 된다.

---

## 🐞 디버깅 3종 세트 (2026-06-29) — 오늘의 핵심

**에러 났을 때 원칙: Swagger 화면 말고, 서버 터미널의 빨간 Traceback 맨 아랫줄을 읽어라.** 거기에 답이 있다.

### 버그 1 — 500: `orm_mode` (Pydantic v1 → v2 이름 변경)

- **증상:** POST/GET 시 500 Internal Server Error
- **원인:** `schemas.py`의 `class Config: orm_mode = True`. Pydantic이 v1→v2 되면서 이 스위치 이름이 `from_attributes`로 바뀜. 옛 이름은 그냥 무시돼서 DB 객체 → JSON 변환이 안 됨.
- **이 스위치가 하는 일:** DB에서 꺼낸 객체(SQLAlchemy 모델)의 속성(`.id`, `.name`)을 읽어서 응답 JSON을 만들어도 된다고 허락.
- **해결:** `orm_mode = True` → `from_attributes = True` (2군데)

### 버그 2 — 500: `UndefinedColumn` (DB 스키마 불일치)

- **증상:** `psycopg2.errors.UndefinedColumn: "memory_hint" 칼럼은 "children" 릴레이션에 없음`
- **원인:** `models.py`엔 `memory_hint` 등 새 칸을 추가했지만, **실제 Postgres 테이블은 그 칸이 추가되기 전 옛날 버전 그대로.**
- **왜 자동으로 안 생기나:** `Base.metadata.create_all()`은 **"없는 테이블만 새로"** 만들 뿐, **이미 있는 테이블에 칸을 덧붙이지 않는다.** → 모델만 바뀌고 DB는 옛날 그대로 = 스키마 불일치.
- **해결 (연습 데이터라 가능):** `drop_all()`로 테이블 싹 지우고 → `create_all()`로 다시 생성. 새 칸 포함해서 재생성됨.

```
from database import engine, Base
import models
Base.metadata.drop_all(bind=engine)    # 기존 테이블 + 데이터 전부 삭제
Base.metadata.create_all(bind=engine)  # 모델대로 새로 생성
```

- **실무에선:** 데이터를 지키면서 칸만 추가하는 **마이그레이션 도구(Alembic)**를 쓴다. (나중에 배움)

### 버그 3 — 404/안 보임: 라우터 경로 오타

- **증상:** `GET /lesson/{id}/lesson-records`가 Swagger에 안 보임
- **원인:** 경로 문자열 오타 `"/{child_id}/leasson-records"` (leasson)
- **해결:** `leasson` → `lesson`. URL 경로는 문자 하나만 틀려도 다른 주소가 된다.

---

## 📌 id vs child_id (기본키 / 외래키) — 2026-06-29

- `id` = 그 표(테이블)에서 **자기 자신의 고유 번호.** 모든 테이블에 있음 → **기본키(primary key)**

- `child_id` = "이 기록이 **어느 아이 거냐**"를 가리키려고 children의 `id`를 베껴 적은 칸 → **외래키(foreign key)**

```
children 표                lesson_records 표
| id | name |             | id | child_id | lesson_date |
|  1 | 유리 |  ◀──────── |  1 |    1     |    6/25     |
                          |  2 |    1     |    6/26     |
```

- `GET /lesson/1/lesson-records` = "**child_id가 1인** 수업기록 다 줘" → 유리(1번)의 기록 5개
- 다른 아이(child_id 2~5)는 기록을 안 넣었으면 빈 리스트 `[]` (에러 아님, "기록 없음")